

Operational Trust Infrastructure for Autonomous Systems

The control layer between autonomous intelligence
and real-world action.

Executive abstract

WHO is acting? · WHAT was delegated? · IS authority still valid? · SHOULD this action execute? · CAN the enterprise prove why?

Enterprises are moving from software that recommends to software that acts. AI agents can call tools, change repositories, move data, operate cloud services, and initiate regulated workflows. Existing identity and access controls remain essential, but they usually answer a narrower question: may this principal access this resource? Consequential autonomy requires an additional, time-sensitive question: is this identified agent currently authorized to perform this exact action, for this purpose, against this target, with adequate evidence and oversight?

Cyber Sentinels provides that decision boundary as an API-first control and evidence layer. A client creates a scoped API key, registers an agent, proves control of an Ed25519 credential, receives a bounded authority grant from an authorized human or service principal, and requests a fresh evaluation for each consequential action. The canonical result is ALLOW, REVIEW, or DENY. The platform persists a transaction record, evidence references, receipt, and Replay view so an operator can reconstruct why the result was reached.

The design keeps important claims separate. Registration is not verification. Verification is not authority. Previous authorization is not standing permission. A provider assertion is not an independent Cyber Sentinels decision. A control plane report is not confirmed downstream outcome evidence. These boundaries make the system useful beside—not in place of—identity providers, policy engines, cloud controls, ticketing systems, and agent orchestration platforms.

V1 is a working Production API with scoped API-key authentication, agent identity proof, bounded authority, consequence-time decisions, receipts, Replay, Trust Memory persistence, tenant isolation, key rotation, revocation, idempotency, and rate limiting. Human review supports approve or reject resolution. Some provider integrations and advanced response automation remain partial or roadmap items; the capability matrix in this paper states those limits explicitly.

The shift from access control to operational trust

Traditional access control establishes a durable relationship between a principal and a resource. That model is necessary, but an autonomous system may hold valid credentials while its current request is unsafe, outside the operator’s intent, based on stale evidence, or aimed at an unexpected target. The risk is not merely unauthorized access. It is an authorized-looking action with the wrong purpose, timing, consequence, or evidence basis.

The product shift is visible in four stages: chat → copilots → agents → autonomous operations. Value increasingly comes from turning complex operational data into real actions across finance, cybersecurity, customer operations, supply chains, cloud infrastructure, HR, procurement, industrial processes, and regulated workflows. Action creates the authority and accountability problem that output-only controls cannot solve.

Operational trust adds a contextual decision point immediately before a consequential action. It combines identity continuity, bounded authority, requested action, environment, target, policy, evidence freshness, and human-oversight requirements. The result applies to one transaction; it does not become a permanent reputation score or universal permission.

This changes the control question:

Access-centric question	Operational-trust question
Can this credential reach the service?	Is this verified agent authorized for this exact action now?
Does a role contain a permission?	Does the current authority grant cover action, target, purpose, environment, and time?
Was a request authenticated?	Is identity continuity supported by current evidence?
Did the control plane report success?	What independent evidence supports execution or outcome?

Cyber Sentinels does not replace IAM, PAM, cloud policy, or agent orchestration. It connects their signals to a canonical, auditable consequence-time decision.

The operational trust problem

AI-agent operations create four linked assurance problems.

First, identity: a name in an API request does not prove control of an agent credential. Second, authority: API access does not allow an agent to enlarge its own mandate. Third, admissibility: an authority that was valid earlier may be revoked, expired, or irrelevant to a later action. Fourth, accountability: an authorization decision must remain reconstructable without pretending it proves downstream execution.

These problems are often split among several systems. An identity provider authenticates a user, an orchestration layer invokes tools, a policy engine evaluates rules, and a SIEM records events. Each system holds part of the story. The control plane can therefore become the only system asserting that its own control worked.

Cyber Sentinels introduces an independent record across that boundary. It binds a registered operational entity to cryptographic proof, binds authority to a scope and version, evaluates the current request, and records a receipt with stable identifiers. Provider evidence can inform the result but cannot silently become the canonical result. Client claims cannot upgrade their own trust level.

The governing principle is simple:

The control plane should not be the only system proving that its own control worked.

How Cyber Sentinels works

01 ACTOR	02 IDENTITY	03 AUTHORITY	04 TRUST DECISION	05 CONTROL	06 EVIDENCE
Human AI agent Service	Credential Key proof Provider	Delegator Scope Expiry	Policy Context Memory	ALLOW REVIEW DENY	Receipt Replay Memory

The V1 lifecycle is deliberately explicit:

- API client — an owner creates a tenant- and client-bound key with least-privilege scopes, an environment, and optional expiry.
- Agent registration — the client registers the operational entity and its declared runtime context.
- Credential and manifest — the agent submits an Ed25519 public credential and a signed manifest.
- Challenge and proof — the service issues a one-time challenge; the agent signs the canonical payload to prove key possession.
- Authority grant — an authorized administrator grants bounded, versioned, expiring authority. The agent cannot self-grant.
- Consequence-time evaluation — every proposed consequential action is evaluated against current identity, authority, policy, and evidence.
- Decision — the API returns ALLOW, REVIEW, or DENY, plus reason codes and stable references.
- Evidence and outcome lineage — clients and providers may append evidence and reported outcomes without rewriting the canonical decision.
- Receipt, Replay, and Trust Memory — persisted views preserve the transaction's decision and evidence history.

Revocation is part of the lifecycle, not an administrative afterthought. When an authority or API key is revoked, later requests must fail safely. A prior ALLOW never creates standing authorization.

API-first commercial surface

The external API is the primary V1 product. It is described by an OpenAPI 3.1 contract and grouped around agent identity, authority, decisions, evidence, transactions, receipts, Replay, reviews, outcomes, and trust state.

The current Production contract exposes 19 paths and 20 operations. Core routes include:

Capability	Representative route
Agent registration and read	POST /agents, GET /agents/{agentId}
Credentials and manifest	POST /agents/{agentId}/credentials, POST /agents/{agentId}/manifest
Challenge and proof	POST /agents/{agentId}/challenge, POST /agents/{agentId}/proof
Authority	POST/GET /agents/{agentId}/authorities, POST .../revoke
Decision	POST /trust/decisions
Evidence	POST /evidence
Transaction, receipt, Replay	GET /trust/transactions/{id}, GET .../receipt, GET .../replay
Human review	GET /reviews/{reference}, POST .../resolve
Outcome lineage	POST /trust/transactions/{id}/outcomes

Requests use scoped API keys in server-to-server contexts. Browser session cookies are not accepted as external API authentication. Idempotency keys make safe retries possible without silently creating a second canonical transaction.

API authentication and client boundaries

An API key is shown once at creation. The service persists a keyed hash and non-secret metadata, not the recoverable raw key. Metadata includes tenant, client, scopes, environment, status, creation time, optional expiry, rotation lineage, and last-used information.

Authentication and authorization are separate. A valid key authenticates an API client; its scopes only permit calls to API capabilities. A key with `authority:write` may be used by an already authorized owner or administrator to manage grants, but the scope alone does not create authority over an agent's real-world actions.

The gateway rejects missing, malformed, invalid, expired, inactive, and revoked credentials. Rotation creates new shown-once material, records lineage, and revokes the replaced key. Tenant and client bindings are applied at persistence and read boundaries so an identifier learned from another tenant does not become readable merely because it is syntactically valid.

Operational guidance:

- store raw keys only in a server-side secret manager;
- issue different keys for environments and integrations;
- grant only the scopes required by that workload;
- rotate before expiry and revoke immediately on suspected exposure;
- never place keys in browser code, URLs, logs, screenshots, or support tickets.

06 · Agent identity and proof

Cyber Sentinels models an AI agent as an operational entity with declared ownership, runtime, and model context. Registration creates the record but leaves identity unverified. This is a critical invariant: `REGISTERED != VERIFIED`.

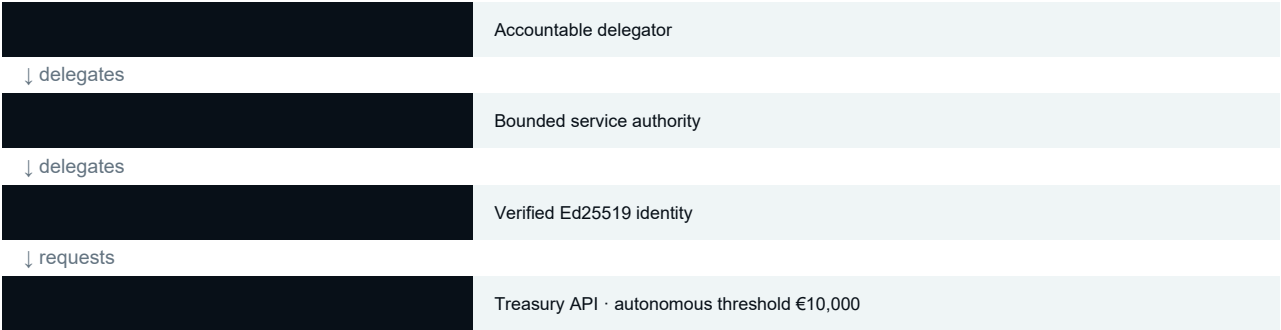
Verification uses possession of an Ed25519 private key. The client registers the public credential and submits a signed manifest. The service then issues a time-bounded, one-time challenge whose canonical payload includes the agent and credential context. A valid signature proves control of the corresponding private key at that moment. Challenges cannot be replayed, and a different private key cannot satisfy the proof.

Verification does not grant permission. A verified agent without a covering authority must still receive `REVIEW` or `DENY` for a consequential action. This preserves the second invariant: `VERIFIED != AUTHORIZED`.

The credential record supports rotation and revocation without changing the agent's stable identity. Manifests retain their own version and digest so later decisions can refer to the declared configuration that was evaluated. Private keys never need to enter Cyber Sentinels.

Authority Graph

CONCEPTUAL OPERATIONAL REPRESENTATION



REVIEW

Human approval required: amount exceeds delegated autonomous threshold.

Authority is represented as a bounded, versioned grant rather than a flat role label. A grant connects the accountable grantor, the verified operational entity, permitted actions, targets or resource patterns, purpose, environment, time bounds, and policy context.

The graph answers not only “what permission exists?” but “who granted it, to which verified entity, under what constraints, and which version was current when this transaction was evaluated?” Stable authority references and versions are carried into decisions and receipts.

No agent can approve its own proof or grant its own authority. An API scope cannot widen the permitted action, target, or purpose stored in the grant. Expiry and revocation are checked at consequence time. Post-revocation decisions cannot be ALLOW based on the revoked grant.

Policy and consequence-time authorization

ALLOW Authorized for this context	REVIEW Accountable intervention required	DENY Not authorized
---	--	-------------------------------

Cyber Sentinels evaluates the proposed action when the consequence is about to occur. It does not treat a previous result as a reusable bearer permission. Two invariants capture this:

PREVIOUS_ALLOW != STANDING_AUTHORIZATION

VALID_AUTHORITY_AT_T1 != AUTOMATIC_PERMISSION_AT_T2

The request identifies the operational entity and a structured action containing type, target, purpose, and environment. The evaluator retrieves current identity state, the relevant authority version, policy, and admissible evidence. It produces a canonical decision with machine-readable reason codes and references.

ALLOW means the current request is authorized under the evaluated constraints. REVIEW means a human decision or missing evidence is required before authorization can proceed. DENY means the request is not authorized. None of these states claims that an external action was executed.

Policy versions and correlation identifiers make later reconstruction possible. Idempotent retries return the existing decision for the same semantic request rather than evaluating an accidental duplicate as a new transaction.

Human oversight and review

Human oversight is a first-class branch of the decision lifecycle. When policy or evidence requires accountable intervention, the service returns REVIEW and a review reference. An authorized reviewer can retrieve the review context and resolve it as approved or rejected.

Approval is not an unrestricted override. The resolved review remains associated with the original transaction context, reviewer identity, timestamp, and reason. A later consequential action still requires a new current evaluation. Rejection preserves the non-authorization path and cannot be rewritten into historical ALLOW evidence.

V1 supports approve and reject resolution through the API. More expressive conditional approval—such as “allow only below this value and before this time”—is a product direction rather than a completed V1 claim. Today, bounded conditions belong in the authority and action context evaluated by policy.

This approach keeps humans accountable without making every agent action manually operated. Organizations can reserve review for high-impact requests, incomplete evidence, changed identity state, or policy boundaries.

10 · Evidence Graph and evidence independence

Evidence is attached to the transaction as referenced, typed material. The Evidence Graph relates identity proof, authority, policy, provider observations, client assertions, review activity, and reported outcomes without collapsing them into one undifferentiated confidence score.

Three separations are enforced:

- `AGENT_ASSERTED != INDEPENDENT_EVIDENCE`
- provider evidence is not the canonical decision;
- a control-plane report is not confirmed downstream outcome evidence.

A client may report that it sent a command or observed a result, but it cannot mark that report as independently verified merely by changing a field. Provider namespaces and digests are validated. Evidence carries provenance, observation time, trust classification, and correlation references where available.

The graph is designed for accountable lineage, not for claiming perfect knowledge. Missing evidence stays missing. Conflicting evidence can drive review or denial. An external provider remains authoritative only for the facts within that provider's documented boundary; Cyber Sentinels remains authoritative for its own canonical decision record.

Receipts

DECISION RECEIPT	Persisted · integrity-linked
TRANSACTION	tx_...72d
AGENT	agent:payments
ACTION	payment.create
RESOURCE	treasury-api
AUTHORITY	grant v3 · current
DECISION	REVIEW
REASON	AMOUNT_EXCEEDS_AUTONOMOUS_THRESHOLD
EVIDENCE	identity · authority · policy · context
REPLAY	replay_...91f

A receipt is the stable, machine-readable record of a canonical decision. It binds identifiers and versions needed to verify what the platform decided:

- decision, transaction, receipt, Replay, agent, and correlation identifiers;
- ALLOW, REVIEW, or DENY plus reason codes;
- authority reference and authority version;
- policy identifier and version;
- evidence references and decision timestamp;
- links to the transaction, receipt, and Replay resources.

Receipts are persisted and integrity-linked. They provide tamper-evident lineage within the service's security boundary; they are not described as physically immutable or as a blockchain artifact. A receipt proves the decision Cyber Sentinels recorded. It does not by itself prove command delivery, downstream execution, or real-world outcome.

This distinction allows receipts to support audit and incident reconstruction without becoming an overbroad attestation. When independent outcome evidence exists, it can be linked after the decision while preserving the original authorization record.

Replay

10:21:03	AGENT AUTHENTICATED
	AUTHORITY RESOLVED
	POLICY EVALUATED
	REVIEW
	HUMAN APPROVAL
10:22:15	FRESH EVALUATION · ALLOW

Replay reconstructs the ordered evidence and decision history for a transaction. It is an investigative and accountability view, not a command re-execution feature.

An operator can follow registration and identity state, authority version, policy evaluation, decision, review events, evidence additions, receipt creation, and reported outcome lineage. Stable Replay identifiers make that view addressable through the API and console.

Replay is useful when a team asks:

- Why was this request allowed, reviewed, or denied?
- Which authority version and policy version were active?
- What evidence was available at decision time?
- Was later evidence independent, provider-supplied, or agent-asserted?
- Did revocation occur before or after the request?

Replay does not retroactively change the canonical result. New evidence can extend the lineage while the original event order and references remain visible.

13 · Trust Memory

Trust Memory is the persisted context that allows later evaluations and investigations to reference prior operational trust events without converting history into standing authority. It indexes transaction, evidence, identity, authority, decision, and outcome lineage for bounded retrieval.

Trust Memory supports continuity questions: has this agent rotated credentials, has its authority changed, did similar requests require review, and which evidence sources were present? It is not a universal trust score and does not make a person or machine permanently “trusted.”

The system separates recorded history from current admissibility. A previous ALLOW is historical evidence that a particular request met the constraints then. It cannot authorize a later request. Current identity, authority, policy, and evidence remain decisive.

Retention, residency, and deletion requirements depend on the enterprise deployment and policy. V1 exposes the working persistence and retrieval foundations; customer-specific governance and reporting are configured during deployment rather than claimed as a single universal compliance posture.

Provider-neutral architecture

Cyber Sentinels accepts signals from multiple providers while keeping its canonical decision independent. Adapters normalize evidence into a common provenance model, but normalization does not erase provider-specific meaning or elevate a provider response into authority.

Provider / integration	V1 state	Honest boundary
Supabase	WORKING	Production data and Auth platform; tenant isolation and persistence remain application and database responsibilities.
Cloudflare Turnstile	WORKING	Human-interaction control on exposed auth/request flows; not agent identity proof.
Hopae	PARTIAL	Adapter and verification path implemented when configured; no retained claim here of a completed live Production-provider qualification.
OpenAI	PARTIAL	Optional assistive governance analysis; never the source of canonical authority or the final decision.
World ID	ROADMAP	Registry placeholder; server-side verification is not a completed V1 integration.
Stripe Identity	ROADMAP	Registry placeholder; billing integration, if present elsewhere, does not equal identity verification.

The architecture can accommodate control planes, identity providers, approval systems, and telemetry providers. Their evidence is constrained by source, time, scope, and verification state.

Operational response and enterprise use cases

Cyber Sentinels can inform operational response without claiming universal downstream enforcement. A client or integrated control plane may use DENY to block its own command, REVIEW to pause a workflow, or revocation to prevent future authorization. Arbitrary third-party kill, quarantine, and rollback automation remains integration-dependent.

Representative use cases include:

Financial services. A payment agent requests a €15,000 transfer while its autonomous authority is limited to €10,000. The current decision is REVIEW; the evidence and human resolution remain linked.

Cloud and cybersecurity. An infrastructure agent attempts to change a Production firewall while its grant is limited to staging. The action is outside scope and receives DENY.

Supply chain. An AI detects an inventory shortage and proposes a purchase order. Cyber Sentinels verifies the agent, procurement authority, supplier, amount, environment, and policy before an integrated system chooses whether to execute.

Enterprise data. An agent that normally reads small customer sets requests a mass export. Current V1 policy and authority can require REVIEW or DENY; advanced behavioral anomaly detection based on that history remains roadmap.

V1 is strongest where an enterprise can place an API decision boundary before a consequential action and return outcome evidence afterward.

Security architecture

The V1 security model is layered:

- shown-once API secrets with hashed persistence, scopes, expiry, revocation, rotation lineage, and last-used metadata;
- Ed25519 agent proof using one-time challenges and signed canonical payloads;
- tenant- and client-scoped reads and writes, reinforced by database row-level security where applicable;
- current authority checks with version, scope, expiry, and revocation;
- idempotency controls for consequential write paths;
- bounded rate limiting and safe generic error responses;
- server-only provider credentials and secret material;
- correlation identifiers and structured logs designed to avoid tokens and raw secrets.

Security is a continuous operating responsibility. Production teams should combine Cyber Sentinels with secret management, network controls, least-privilege cloud roles, dependency management, incident response, and monitoring. No whitepaper can substitute for a customer-specific threat model or independent assessment.

The platform fails closed where required identity, authority, scope, or evidence is absent. It does not accept browser cookies as external API authorization and does not permit an agent to assert its way into verified or authorized state.

Data model and deployment architecture

The application is deployed as a Next.js service on Vercel with Supabase providing PostgreSQL and Auth. Cloudflare Turnstile protects selected public human-facing flows. The public API remains accessible through the canonical Production origin.

Canonical records include API clients and keys, operational entities, credentials, manifests, challenges, authority grants, transactions, decisions, evidence nodes and edges, review events, receipts, Replay sessions, and Trust Memory indexes. Stable identifiers connect those records without exposing secret material.

Production migrations are forward-only and reconciled against the live ledger before application. Security-definer database functions use constrained execution grants. Row-level policies and application checks preserve tenant boundaries; service-role use remains server-side.

V1 is delivered as a managed Production service. Private cloud, on-premises, and customer-operated control-plane deployments are product directions, not generally available V1 deployment claims.

Current product reality

The following matrix describes the Production V1 boundary as of this paper's version.

Capability	State	Evidence boundary
External OpenAPI 3.1 API	WORKING	Production contract exposes 19 paths and 20 operations.
Scoped API-key auth, rotation, revocation	WORKING	Raw key shown once; hashed persistence and metadata retained.
Agent registration and Ed25519 proof	WORKING	Registration, credential, manifest, challenge, and proof are distinct states.
Bounded authority and current evaluation	WORKING	Grant, read, expiry, revocation, and post-revocation non-ALLOW behavior.
ALLOW / REVIEW / DENY	WORKING	Result authorizes the evaluated request; it does not prove execution.
Receipt, Replay, Trust Memory	WORKING	Persisted decision and evidence lineage.
Tenant/client isolation, idempotency, rate limiting	WORKING	Qualified API boundaries and safe failure behavior.
Human review approve/reject	WORKING	Resolution is bounded to the referenced review.
Conditional approval workflow	PARTIAL	Conditions can be represented in authority/action context; a dedicated condition-resolution product flow is roadmap.
Hopae provider operation	PARTIAL	Adapter available when configured; live provider proof is deployment-specific.
Operations Console and reporting	PARTIAL	Working transaction, evidence, Replay, review, API-key, and account surfaces; advanced compliance packs remain roadmap.
Authority Graph UI	ROADMAP	The authority model and API are working; a dedicated graph exploration interface is not claimed as V1.
Policy Studio	ROADMAP	V1 evaluates versioned policy; a general-purpose visual authoring studio is not shipped.
Arbitrary third-party block/kill/quarantine	ROADMAP	Requires explicit control-plane integrations and downstream acknowledgement.
Advanced behavioral anomaly intelligence	ROADMAP	Research and product direction; not a V1 authorization claim.
Compliance reporting	PARTIAL	Evidence is retrievable; packaged framework-specific reports remain deployment and roadmap work.
Private/on-prem deployment	ROADMAP	Managed cloud is the current delivery model.

Platform direction

The next phase extends the operational trust boundary without weakening V1's separations.

Near-term direction includes richer conditional human review, more provider adapters with retained live qualification evidence, clearer authority-graph exploration, expanded operational reporting, and control-plane integrations that can return command acknowledgement and independently observed outcome evidence.

Longer-term work includes customer-controlled deployment options, advanced anomaly signals, policy simulation, and stronger verification coverage across heterogeneous agent runtimes. These capabilities must remain explainable and scoped. Model-generated recommendations can assist operators but must not silently replace accountable authority or canonical policy evaluation.

The architecture is designed so new signals enrich evidence rather than redefine truth. Identity providers prove identity within their boundary. Control planes enforce within theirs. Cyber Sentinels records whether a particular verified entity had current bounded authority and what evidence supported the consequence-time decision.

20 · Closing: accountable autonomy

Autonomous systems need more than credentials and retrospective logs. They need a current, accountable decision boundary before consequential action and an independent lineage afterward.

Cyber Sentinels exposes a Production API that verifies AI-agent identity, evaluates bounded current authority for consequential actions, returns ALLOW / REVIEW / DENY, and preserves evidence through Receipt and Replay.

That is the V1 foundation for accountable autonomy: identity without self-assertion, authority without silent expansion, decisions without standing permission, and evidence without pretending that authorization equals execution.

VERIFICATION REFERENCES

www.cybersentinels.com/api/health

www.cybersentinels.com/api/ready

www.cybersentinels.com/api/v1/openapi.json

www.cybersentinels.com/developers/docs